

CS50's Introduction to Programming with R

OpenCourseWare

Donate  (<https://cs50.harvard.edu/donate>)

Carter Zenke (<https://carterzenke.me>)

carter@cs50.harvard.edu

 (<https://github.com/carterzenke>)  (<https://www.linkedin.com/in/carterzenke/>)

David J. Malan (<https://cs.harvard.edu/malan/>)

malan@harvard.edu

 (<https://www.facebook.com/dmalan>)  (<https://github.com/dmalan>) 

(<https://www.instagram.com/davidjmalan/>)  (<https://www.linkedin.com/in/malan/>) 

(<https://www.reddit.com/user/davidjmalan>)  (<https://www.threads.net/@davidjmalan>) 

(<https://twitter.com/davidjmalan>)

Lecture 6

- [Welcome!](#)
- [Exceptions](#)
- [message](#)
- [warning](#)
- [stop](#)
- [Unit Tests](#)
- [testthat](#)
- [Testing Floating-Point Values](#)
- [Tolerance](#)
- [Test-Driven Development](#)
- [Behavior-Driven Development](#)
- [Test Coverage](#)
- [Summing Up](#)

Welcome!

- Welcome back to CS50's Introduction to Programming with R!
- Today, we will be learning about testing programs. We will see how our programs can go wrong, how we can handle things when they do, and how to methodically test our programs to ensure they behave as we expect!

Exceptions

- Consider the following program that calculates an average:

```
# Define function to calculate average value in a vector

average <- function(x) {
  sum(x) / length(x)
}
```

Notice how this program attempts to take as its input a vector of numbers and output the average.

- You can imagine how your user may accidentally pass characters instead of numbers, resulting in our `average` function outputting an error.
- These errors are called *exceptions*. Could there be a way by which we could check for potential such exceptions? Consider the following update to `average`:

```
# Handle non-numeric input

average <- function(x) {
  if (!is.numeric(x)) {
    return(NA)
  }
  sum(x) / length(x)
}
```

Notice how a conditional, an `if` statement, checks to see if the vector `x` is not full of numbers. By convention in the R world, returning a value `NA` is appropriate in such an instance.

message

- While this allows our program to run silently, we may wish to let the user know that an exception has occurred. One way to alert the user is via the `message` function:

```
# Message about returning NA

average <- function(x) {
  if (!is.numeric(x)) {
    message("`x` must be a numeric vector. Returning NA instead.")
    return(NA)
  }
  sum(x) / length(x)
}
```

Notice how a `message` is sent to the user about why the program is returning `NA` instead.

- Traditionally, `message` is intended for when something has *not* gone wrong: `message` is purely for informational purposes. Thus, we can escalate the importance of this

information through a `warning`.

warning

- We can escalate the importance of our `message` to a `warning` as follows:

```
# Warn about returning NA

average <- function(x) {
  if (!is.numeric(x)) {
    warning("`x` must be a numeric vector. Returning NA instead.")
    return(NA)
  }
  sum(x) / length(x)
}
```

Notice how the output is now a warning message.

- A `warning` doesn't stop a program altogether, but it does let a programmer know that something has gone wrong.

stop

- You can imagine situations where you don't simply want to warn the user; You may want to completely stop the function. Consider the following:

```
# Stop instead of warn

average <- function(x) {
  if (!is.numeric(x)) {
    stop("`x` must be a numeric vector.")
  }
  sum(x) / length(x)
}
```

Notice how `stop` tells the user we cannot proceed given the input they have provided us.

- It is also possible to combine both possibilities. For example, the following code looks at both situations where `x` contains non-numeric elements. Similarly, this code accommodates situations where there are `NA` values:

```
# Handle NA values

average <- function(x) {
  if (!is.numeric(x)) {
    stop("`x` must be a numeric vector.")
  }
  if (any(is.na(x))) {
    warning("`x` contains one or more NA values.")
    return(NA)
  }
}
```

```
    sum(x) / length(x)
}
```

Notice how two `if` statements are provided.

Unit Tests

- *Unit tests* are used to test our functions and programs.
- Consider the following test function for `average` in a separate file:

```
# Write test function

source("average6.R")

test_average <- function() {
  if (average(c(1, 2, 3)) == 2) {
    cat("`average` passed test :)\n")
  } else {
    cat("`average` failed test :(\n")
  }
}

test_average()
```

Notice that this function provides a test case where the numbers `1`, `2`, and `3` are provided to the `average` function. Then, some feedback is provided. Notice how, in the first line, `source` ensures this test file has access to the `average` function.

- It would also be wise to test negative numbers:

```
# Add test cases

source("average6.R")

test_average <- function() {
  if (average(c(1, 2, 3)) == 2) {
    cat("`average` passed test :)\n")
  } else {
    cat("`average` failed test :(\n")
  }

  if (average(c(-1, -2, -3)) == -2) {
    cat("`average` passed test :)\n")
  } else {
    cat("`average` failed test :(\n")
  }

  if (average(c(-1, 0, 1)) == 0) {
    cat("`average` passed test :)\n")
  } else {
    cat("`average` failed test :(\n")
  }
}
```

```
test_average()
```

Notice how additional tests are provided for positive and negative numbers and zero.

- We have already written 21 lines of code! Thankfully, programmers have already created various test packages or libraries that can be used to test our code.

testthat

- testthat is a package for testing R code. It can be loaded by typing `library(testthat)` into your console.
- testthat includes a function called `test_that` that can be used to test our function:

```
# Test warning about NA values

source("average6.R")

test_that("`average` calculates mean", {
  expect_equal(average(c(1, 2, 3)), 2)
  expect_equal(average(c(-1, -2, -3)), -2)
  expect_equal(average(c(-1, 0, 1)), 0)
  expect_equal(average(c(-2, -1, 1, 2)), 0)
})

test_that("`average` warns about NAs in input", {
  expect_warning(average(c(1, NA, 3)))
  expect_warning(average(c(NA, NA, NA)))
})
```

Notice how the `test_that` function can be told to expect that that the `average` of various numbers will equal a certain value, thanks to `expect_equal`. Similarly, we can provide the `test_that` function instructions to `expect_warning` when the average calculation includes `NA` values. Further, notice how the test is divided into various sections. One section tests the calculation of the mean, while another tests the warnings.

- Running the above test, we discover that the order of our `if` statements in our `average` function may be out of order:

```
# Fix ordering of error handling

average <- function(x) {
  if (any(is.na(x))) {
    warning("`x` contains one or more NA values.")
    return(NA)
  }
  if (!is.numeric(x)) {
    stop("`x` must be a numeric vector.")
  }
  sum(x) / length(x)
}
```

Notice how the order of the conditional statements is altered.

- We should still test that `average` returns `NA` when given an `NA` value in its input, not just that `average` raises a warning!

```
# Test NA return values

source("average7.R")

test_that("`average` calculates mean", {
  expect_equal(average(c(1, 2, 3)), 2)
  expect_equal(average(c(-1, -2, -3)), -2)
  expect_equal(average(c(-1, 0, 1)), 0)
  expect_equal(average(c(-2, -1, 1, 2)), 0)
})

test_that("`average` returns NA with NAs in input", {
  expect_equal(suppressWarnings(average(c(1, NA, 3))), NA)
  expect_equal(suppressWarnings(average(c(NA, NA, NA))), NA)
})

test_that("`average` warns about NAs in input", {
  expect_warning(average(c(1, NA, 3)))
  expect_warning(average(c(NA, NA, NA)))
})
```

Notice how we have two separate tests that pass `NA` values as input to `average`. One tests for the right return value, while the other tests for a `warning` to be raised.

- `test_that` has other functions that can assist us in testing, including `expect_error` and `expect_no_error`.
- Using `expect_error` we can modify our code as follows:

```
# Test stop if argument is non-numeric

source("average7.R")

test_that("`average` calculates mean", {
  expect_equal(average(c(1, 2, 3)), 2)
  expect_equal(average(c(-1, -2, -3)), -2)
  expect_equal(average(c(-1, 0, 1)), 0)
  expect_equal(average(c(-2, -1, 1, 2)), 0)
})

test_that("`average` returns NA with NAs in input", {
  expect_equal(suppressWarnings(average(c(1, NA, 3))), NA)
  expect_equal(suppressWarnings(average(c(NA, NA, NA))), NA)
})

test_that("`average` warns about NAs in input", {
  expect_warning(average(c(1, NA, 3)))
  expect_warning(average(c(NA, NA, NA)))
})

test_that("`average` stops if `x` is non-numeric", {
  expect_error(average(c("quack!")))
  expect_error(average(c("1", "2", "3")))
})
```

Notice how this code expects an error when the input is “quack!” or when characters are provided instead of numbers.

Testing Floating-Point Values

- We may wish to provide floating-point values (i.e., decimal values) as input to `average`:

```
# Test doubles

source("average7.R")

test_that("`average` calculates mean", {
  expect_equal(average(c(1, 2, 3)), 2)
  expect_equal(average(c(-1, -2, -3)), -2)
  expect_equal(average(c(-1, 0, 1)), 0)
  expect_equal(average(c(-2, -1, 1, 2)), 0)
  expect_equal(average(c(0.1, 0.5)), 0.3)
})

test_that("`average` returns NA with NAs in input", {
  expect_equal(suppressWarnings(average(c(1, NA, 3))), NA)
  expect_equal(suppressWarnings(average(c(NA, NA, NA))), NA)
})

test_that("`average` warns about NAs in input", {
  expect_warning(average(c(1, NA, 3)))
  expect_warning(average(c(NA, NA, NA)))
})

test_that("`average` stops if `x` is non-numeric", {
  expect_error(average(c("quack!")))
  expect_error(average(c("1", "2", "3")))
})
```

Notice how a test for floating-point values is added at the end of the first set of tests.

Tolerance

- Floating-point values are unique, in that they are subject to floating-point imprecision.
- Let's understand floating-point imprecision by example:

```
# Demonstrates floating-point imprecision

print(0.3)
print(0.3, digits = 17)
```

Notice how we see that 0.3 is not represented as precisely 0.3 in R. This is a common phenomenon across programming languages, given that there are an infinite number of floating-point values and a finite number of bits to represent them.

- Because of floating-point imprecision, tests of equality involving floating-point values need to allow for some *tolerance*. Tolerance refers to a range of values, above or below the

expected value, that will be considered—for the sake of the test—to be equal to the expected value. Tolerance is often specified in absolute terms, such as $\pm .000001$.

- The `expect_equal` function already provides a level of tolerance that is generally acceptable for most use cases. This default can be changed with the `tolerance` argument.
- You and your team should decide upon what level of precision is expected in your calculations.

Test-Driven Development

- One philosophy of development is called *test-driven development*. In this mindset, the belief is that it is best to create a test first before even writing the source code that will be tested. Consider the following test:

```
# Test greet

source("greet1.R")

test_that("`greet` says hello to a user", {
  expect_equal(greet("Carter"), "hello, Carter")
})
```

Notice how you can imagine that a `greet` function should greet a user provided as input.

- Looking at the test, we could create code that responds to the test:

```
# Greet a user

greet <- function(to) {
  return(paste("hello,", to))
}
```

Notice how this code says `hello` to a user by name.

- In test-driven development, writing tests allows programmers to know what functionality they should implement. The benefit is that this functionality is then immediately testable. Further modifications should always pass the tests one has already written.

Behavior-Driven Development

- *Behavior-driven development* is similar in spirit to test-driven development, with a greater focus on the behavior of a function in context. In behavior-driven development, one might describe what we want the function to do by explicitly naming what it should do.
- `testthat` comes with two functions to implement behavior-driven development, `describe` and `it`:


```
# Describe greet

source("greet2.R")

describe("greet()", {
  it("can say hello to a user", {
    name <- "Carter"
    expect_equal(greet(name), "hello, Carter")
  })
  it("can say hello to the world", {
    expect_equal(greet(), "hello, world")
  })
})
```

Notice how `describe` includes several code-based descriptions of what `it` (the function!) should be able to do.

Test Coverage

- As you go off and write tests for your code, consider how comprehensive these tests are. Define what is critical for your code to accomplish and create tests that exemplify those critical tasks.

Summing Up

In this lesson, you learned how to test programs in R. Specifically, you learned about:

- Exceptions
 - `message`
 - `warning`
 - `stop`
- Unit Tests
- `testthat`
- Testing Floating-Point Values
- Tolerance
- Test-Driven Development
- Behavior-Driven Development
- Test Coverage

See you next time when we can package our code and share it with the world.

